Assignment No. 03

Title: AVL Tree with Rotations. Implement insertion and deletion operations in an AVL Tree. After each update, identify and display the rotation performed (LL, LR, RL, or RR).

Introduction:

AVL Tree:

- An **AVL tree** is a self-balancing Binary Search Tree where the height difference between left and right subtrees (balance factor) of every node is at most 1.
- After every insertion or deletion, rotations are performed to maintain balance, ensuring all operations (search, insert, delete) run in **O(log n)** time.

Structure of AVL Tree:

Each node in an AVL tree contains:

- **Data** (key value)
- **Left child pointer**
- **Right child pointer**
- **Height** (important for balance calculation)

So compared to a normal BST, AVL stores one extra field: height.

Types of Rotations Performed

1. **LL Rotation (Right Rotation):**
   Performed when a node becomes unbalanced due to insertion/deletion in the left subtree of its left child.
2. **RR Rotation (Left Rotation):**
   Performed when a node becomes unbalanced due to insertion/deletion in the right subtree of its right child.
3. **LR Rotation (Left-Right Rotation):**
   Performed when imbalance occurs in the right subtree of the left child, so first left rotation on left child then right rotation on root.
4. **RL Rotation (Right-Left Rotation):**
   Performed when imbalance occurs in the left subtree of the right child, so first right rotation on right child then left rotation on root.

Pseudocodes:

Insertion –

```
FUNCTION INSERT(root, data)
1. IF root == NULL THEN
      RETURN CREATE(data)
2. IF data < root.data THEN
      root.left = INSERT(root.left, data)
3. ELSE IF data > root.data THEN
      root.right = INSERT(root.right, data)
4. b = BALANCE(root)
   // LL Case
5. IF b > 1 AND data < root.left.data THEN
```

```
        RETURN LL(root)
    // RR Case
6. IF b < -1 AND data > root.right.data THEN
        RETURN RR(root)
    // LR Case
7. IF b > 1 AND data > root.left.data THEN
        root.left = RR(root.left)
        RETURN LL(root)
    // RL Case
8. IF b < -1 AND data < root.right.data THEN
        root.right = LL(root.right)
        RETURN RR(root)
9. root.height = 1 + MAX(HEIGHT(root.left), HEIGHT(root.right))
10. RETURN root
```

Deletion –

```
FUNCTION DELETE(root, data)
1. IF root == NULL THEN
        RETURN root
2. IF data < root.data THEN
        root.left = DELETE(root.left, data)
3. ELSE IF data > root.data THEN
        root.right = DELETE(root.right, data)
4. ELSE   // Node found
        IF root.left == NULL THEN
            temp = root.right
            DELETE root
            RETURN temp
        ELSE IF root.right == NULL THEN
            temp = root.left
            DELETE root
            RETURN temp
        temp = MIN_VALUE_NODE(root.right)
        root.data = temp.data
        root.right = DELETE(root.right, temp.data)
5. IF root == NULL THEN
        RETURN root
6. root.height = 1 + MAX(HEIGHT(root.left), HEIGHT(root.right))
7. b = BALANCE(root)
    // LL Case
8. IF b > 1 AND BALANCE(root.left) >= 0 THEN
        RETURN LL(root)
    // LR Case
9. IF b > 1 AND BALANCE(root.left) < 0 THEN
        root.left = RR(root.left)
        RETURN LL(root)
    // RR Case
10. IF b < -1 AND BALANCE(root.right) <= 0 THEN
         RETURN RR(root)
    // RL Case
```

11. IF b < -1 AND BALANCE(root.right) > 0 THEN
    root.right = LL(root.right)
    RETURN RR(root)
12. RETURN root

Algorithms:

Insertion -

Step 1: If root is NULL, create a new node with given value and return it.
Step 2: If value is less than root.data, recursively insert into the left subtree.
Step 3: Else if value is greater than root.data, recursively insert into the right subtree.
Step 4: Update the height of the current node.
Step 5: Calculate Balance Factor
Balance = height(left subtree) − height(right subtree)
Step 6: If Balance > 1 and value < left child data
Perform Right Rotation (LL Case).
Step 7: If Balance < −1 and value > right child data
Perform Left Rotation (RR Case).
Step 8: If Balance > 1 and value > left child data
Perform Left Rotation on left child, then Right Rotation (LR Case).
Step 9: If Balance < −1 and value < right child data
Perform Right Rotation on right child, then Left Rotation (RL Case).
Step 10: Return the updated root.

Deletion -

Step 1: If root is NULL, return NULL.
Step 2: If value is less than root.data, delete from left subtree.
Step 3: Else if value is greater than root.data, delete from right subtree.
Step 4: Else (node found):

- If node has no child, Delete the node.
- If node has one child, Replace node with its child.
- If node has two children:
    1. Find inorder successor (smallest in right subtree).
    2. Copy successor value to current node.
    3. Delete the successor.

Step 5: Update the height of current node.
Step 6: Calculate Balance Factor.
Step 7: If Balance > 1 and left subtree balance ≥ 0
Perform Right Rotation (LL Case).
Step 8: If Balance > 1 and left subtree balance < 0
Perform Left Rotation on left child, then Right Rotation (LR Case).
Step 9: If Balance < −1 and right subtree balance ≤ 0
Perform Left Rotation (RR Case).
Step 10: If Balance < −1 and right subtree balance > 0
Perform Right Rotation on right child, then Left Rotation (RL Case).
Step 11: Return updated root.

Flowcharts:

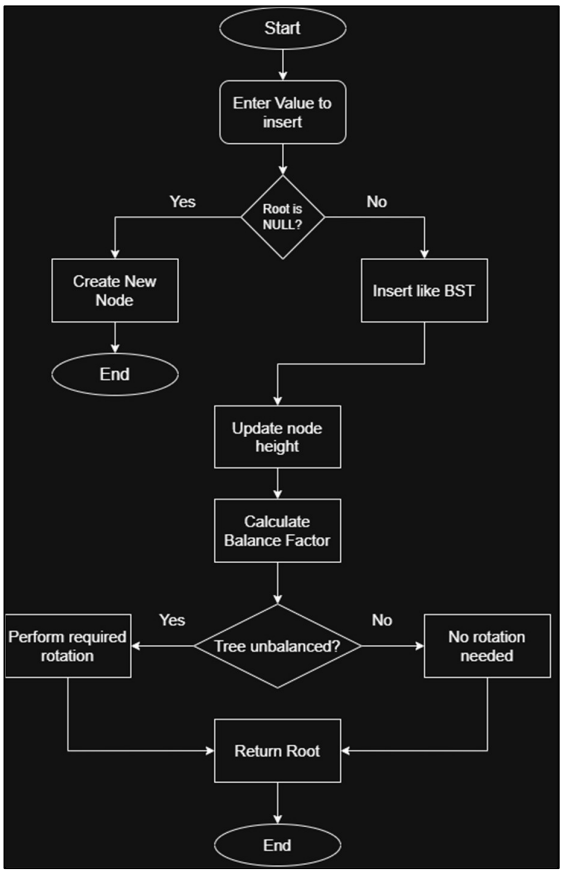

Fig 1 – Insertion in AVL Tree

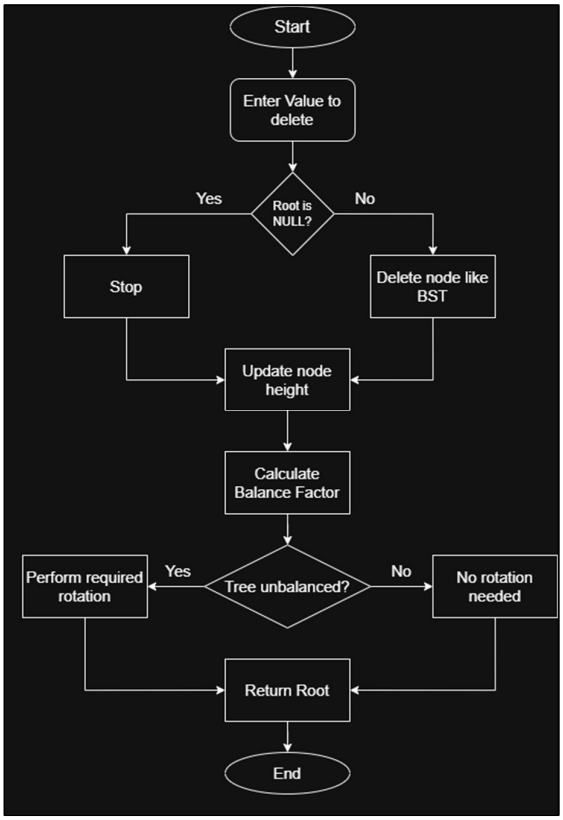

Fig 2 – Deletion in AVL Tree

Codes:

AVLTree.cpp

```
#include <iostream>
using namespace std;
struct Node{
    public:
    int data;
    Node* l;
    Node* r;
    int height;
};
Node* create(int data){
    Node* root=new Node();
    root->data=data;
    root->l=NULL;
    root->r=NULL;
    root->height=1;
    return root;
}
int height(Node* root){
    return root==NULL ? 0 : root->height;
}
int balance(Node* root){
    return root==NULL ? 0 : height(root->l)-height(root->r);
}
Node* LL(Node* root){
    Node* x=root->l;
    Node* y=x->r;

    x->r=root;
    root->l=y;

    root->height=max(height(root->l),height(root->r))+1;
    x->height=max(height(x->l),height(x->r))+1;
    return x;
}
Node* RR(Node* root){
    Node* x=root->r;
    Node* y=x->l;

    x->l=root;
    root->r=y;

    root->height=max(height(root->l),height(root->r))+1;
    x->height=max(height(x->l),height(x->r))+1;
    return x;
}
Node* insert(Node* root,int data){
    if(root==NULL){
        return create(data);
    }
    if(data<root->data){
        root->l=insert(root->l,data);
    }else if(data>root->data){
        root->r=insert(root->r,data);
    }
    int b=balance(root);

    if(b>1&&data<root->l->data){
```

```
        cout << "Performing LL (Right) rotation" << endl;
        return LL(root);
    }

    if(b<-1&&data>root->r->data){
        cout << "Performing RR (Left) rotation" << endl;
        return RR(root);
    }

    if(b>1&&data>root->l->data){
        cout << "Performing LR rotation" << endl;
        root->l=RR(root->l);
        return LL(root);
    }

    if(b<-1&&data<root->r->data){
        cout << "Performing RL rotation" << endl;
        root->r=LL(root->r);
        return RR(root);
    }
    root->height = 1 + max(height(root->l), height(root->r));
    return root;
}
Node* minval (Node* node){
    Node* curr = node;
    while(curr->l != NULL){
        curr = curr->l;
    }
    return curr;
}
Node* deleteN (Node* root, int data){
    if(root == NULL) return root;
    if(data < root->data){
        root->l = deleteN(root->l, data);
    }else if(data > root->data){
        root->r = deleteN(root->r, data);
    }else{
        if(root->l == NULL){
            Node* temp = root->r;
            delete root;
            return temp;
        }else if(root->r == NULL){
            Node* temp = root->l;
            delete root;
            return temp;
        }
        Node* temp = minval(root->r);
        root->data = temp->data;
        root->r = deleteN (root->r, temp->data);
    }
    if(root == NULL) return root;
    root->height = 1 + max(height(root->l), height(root->r));
    int b = balance(root);
    if(b > 1 && balance(root->l) >= 0){
        cout << "Performing LL (Right) rotation after deletion" << endl;
        return LL(root);
    }
    if(b > 1 && balance(root->l) < 0){
        cout << "Performing LR rotation after deletion" << endl;
```

```
        root->l = RR(root->l);
        return LL(root);
    }
    if(b < -1 && balance(root->r) <= 0){
        cout << "Performing RR (Left) rotation after deletion" << endl;
        return RR(root);
    }
    if(b < -1 && balance(root->r) > 0){
        cout << "Performing RL rotation after deletion" << endl;
        root->r = LL(root->r);
        return RR(root);
    }
    return root;
}
void display(Node* root){
    if(root!=NULL){
        display(root->l);
        cout<<root->data<<" ";
        display(root->r);
    }
}
int main(){
    int data;
    int c=0;
    Node* root=NULL;
    while(c!=4){
        cout<<"Menu :"<<endl;
        cout<<"1. Insert"<<endl;
        cout<<"2. Delete"<<endl;
        cout<<"3. Display"<<endl;
        cout<<"4. Exit"<<endl;
        cout<<"Enter your choice: "<<endl;
        cin>>c;
        switch(c){
            case 1:
                cout<<"Enter data to insert: "<<endl;
                cin>>data;
                root = insert(root, data);
                break;
            case 2:
                cout<<"Enter data to delete: "<<endl;
                cin>>data;
                root = deleteN(root, data);
                break;
            case 3:
                cout<<"AVL Tree: "<<endl;
                display(root);
                cout<<endl;
                break;
            case 4:
                cout<<"Bye Bye"<<endl;
                break;
            default:
                cout<<"Choose correct option"<<endl;
        }
    }
}
```

Outputs:

AVL Tree Insertion

```
Menu :
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
1
Enter data to insert:
10
Performing LL (Right) rotation
Menu :
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
1
Enter data to insert:
8
Menu :
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
1
Enter data to insert:
4
Performing LL (Right) rotation
```

Display

```
Enter your choice:
3
AVL Tree:
3 4 5 8 10 12 15 18 20
```

AVL Tree Deletion

```
Menu :
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
12
Choose correct option
Menu :
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
2
Enter data to delete:
8
Performing RR (Left) rotation after deletion
```